

SAAS & WEB

SaaS Fullstack Mastery: De 0 à Produção

Construa uma plataforma SaaS completa do zero: React/Next.js 14, Node.js, arquitetura multi-tenant, assinaturas com Stripe e MercadoPago.

[**Nível: Intermediário**]

[**Duração: 24h**]

[**16 Módulos**]

[**Tech: Next.js 14, Node.js, MercadoPago, PostgreSQL**]

#> Módulo 1: Arquitetura SaaS

O Software as a Service (SaaS) revolucionou a forma como distribuimos aplicativos. Em vez de vender licenças de uso único, cobramos assinaturas recorrentes.

O Desafio Multi-Tenant

Em um SaaS, centenas de empresas (inquilinos/tenants) usam o mesmo aplicativo simultaneamente. O maior desafio de arquitetura é garantir que a Empresa A nunca possa ver, por acidente, os dados da Empresa B, embora ambos compartilhem o mesmo Banco de Dados.

Nossa Pilha de Tecnologia

Para resolver isso, construiremos uma pilha moderna:

- Frontend/Backend: Next.js 14 (App Router) permite SEO, interface rápida e código de servidor em um único projeto.
- Banco de Dados: PostgreSQL por sua robustez em dados relacionais.
- ORM: Prisma ou Drizzle para tipagem segura.
- Pagamentos: MercadoPago (América Latina) e Stripe (Global).

#> Módulo 2: Configuração do Ambiente

Vamos começar nosso projeto do zero. Usaremos as ferramentas oficiais.

Passo 1: Inicializar o Next.js 14

1. Abra seu terminal.
2. Execute o comando para criar o projeto base. Ele nos fará várias perguntas.

```
npx create-next-app@latest meu-saas
# Selecione:
# TypeScript: Yes
# ESLint: Yes
# Tailwind CSS: Yes
# `src/` directory: Yes
# App Router: Yes
```

3. Entre na pasta com ``cd meu-saas`` e abra-a no seu editor com ``code .``

Passo 2: Banco de Dados Local

Em vez de instalar o PostgreSQL nativo que pode quebrar seu sistema, usaremos Docker para rodá-lo de forma limpa.

```
# Crie um arquivo docker-compose.yml na raiz:
version: '3.8'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: root
      POSTGRES_PASSWORD: password123
      POSTGRES_DB: saas_db
    ports:
      - "5432:5432"
```

Execute ``docker-compose up -d`` em seu terminal para levantar o banco de dados em segundo plano.

#> Módulo 3: Modelagem de Dados (Prisma ORM)

Um ORM nos permite escrever código JavaScript/TypeScript em vez de SQL puro, autocompletando erros antes que aconteçam.

Passo 1: Instalação do Prisma

1. Instale o Prisma: ``npm install prisma --save-dev`` e ``npm install @prisma/client``.
2. Inicialize o Prisma: ``npx prisma init``.

Passo 2: O Esquema Multi-Tenant

Abra o arquivo recém-criado ``prisma/schema.prisma``. Definiremos nosso modelo fundamental: Organizações (Tenants) e Usuários.

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

model Tenant {
  id          String    @id @default(uuid())
  name        String
  stripeId    String?   // ID do cliente no Stripe
  plan        String    @default("FREE")
  users       User[]
  createdAt   DateTime  @default(now())
}

model User {
  id          String    @id @default(uuid())
  email       String    @unique
  name        String?
  password    String
  tenantId    String
```


#> Módulo 4: Autenticação com NextAuth

A segurança não deve ser reinventada. Usaremos NextAuth.js (Auth.js) para lidar com sessões criptografadas e login seguro.

Passo 1: Instalação e Configuração

Instale `npm install next-auth`.

```
// Crie o arquivo: src/app/api/auth/[...nextauth]/route.ts
import NextAuth from "next-auth"
import CredentialsProvider from "next-auth/providers/credentials"
import { prisma } from "@/lib/prisma"

const handler = NextAuth({
  providers: [
    CredentialsProvider({
      name: "Credentials",
      credentials: {
        email: { label: "Email", type: "text" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        // 1. Buscar usuário no BD
        const user = await prisma.user.findUnique({ where: { email: credentials?.email } })

        if (!user) return null

        // 2. Verificar senha (em prod, use bcrypt.compare)
        if (user.password !== credentials?.password) return null

        // 3. Retornar dados, injetando o Tenant ID
        return { id: user.id, name: user.name, email: user.email, tenantId: user.tenantId }
      }
    })
  ],
  callbacks: {
    session({ session, token }) {
      // Injetar o tenantId na sessão pública
      if (token && session.user) {
        session.user.tenantId = token.tenantId
      }
      return session
    }
  }
})
```


#> Módulo 5: UI/UX com Tailwind e Shadcn UI

Seu SaaS deve parecer premium. Não escreveremos CSS à mão. Usaremos TailwindCSS (já instalado) e Shadcn UI, uma coleção de componentes acessíveis e bonitos.

Passo 1: Inicializar Shadcn

1. No terminal: ``npx shadcn-ui@latest init``
2. Ele pedirá para escolher um estilo (Default ou New York) e cores. Escolha seus favoritos.
3. Instale um botão: ``npx shadcn-ui@latest add button``.

Passo 2: Sua primeira view

```
// src/app/page.tsx
import { Button } from "@/components/ui/button"

export default function LandingPage() {
  return (
    <main className="flex min-h-screen flex-col items-center justify-center bg-zinc-950 text-white p-24">
      <h1 className="text-5xl font-bold tracking-tight mb-6">
        O SaaS do Futuro
      </h1>
      <p className="text-zinc-400 mb-8 max-w-lg text-center">
        Otimize seus processos de negócios com nossa plataforma impulsionada por IA.
      </p>
      <Button size="lg" className="bg-emerald-500 hover:bg-emerald-600">
        Começar Teste Grátis
      </Button>
    </main>
  )
}
```

Ao salvar, o Tailwind aplica as classes instantaneamente e Shadcn fornece um botão com animações e acessibilidade perfeitas.

#> Módulo 6: Protegendo o Dashboard

O App Router do Next.js nos permite validar o usuário no servidor antes de sequer enviar o HTML para o navegador, melhorando a segurança e a velocidade.

Criando uma rota protegida

Crie a pasta `src/app/dashboard/page.tsx`.

```
import { getServerSession } from "next-auth"
import { redirect } from "next/navigation"
import { prisma } from "@/lib/prisma"

export default async function DashboardPage() {
  // 1. Obter a sessão no SERVIDOR
  const session = await getServerSession()

  // 2. Proteger a rota: Se não houver sessão, chutar o usuário
  if (!session || !session.user) {
    redirect("/api/auth/signin")
  }

  // 3. Consultar dados filtrando pelo TENANT do usuário
  const tenantId = session.user.tenantId
  const tenantData = await prisma.tenant.findUnique({
    where: { id: tenantId }
  })

  return (
    <div className="p-8">
      <h1 className="text-3xl font-bold">Painel de Controle</h1>
      <p>Bem-vindo de volta, {session.user.name}</p>
      <div className="mt-4 p-4 border rounded">
        Empresa atual: {tenantData?.name}
        Plano: {tenantData?.plan}
      </div>
    </div>
  )
}
```

Observe como NUNCA enviamos o tenantId do cliente! Lemos do cookie criptografado da sessão. Isso evita que um hacker altere o ID para ver dados de outra empresa.

#> Módulo 7 a 16: Integrações Finais

Nas fases finais do curso, nos aprofundaremos nas interações do mundo real com seu SaaS.

Stripe e MercadoPago (Módulos 8 a 10)

Você configurará ambos os SDKs. O Stripe é perfeito para assinaturas globais com seu portal de cliente pré-construído. O MercadoPago é necessário se seus usuários preferem pagar com moedas locais ou Pix. O mais crucial aqui é construir Webhooks: Endpoints que escutam os servidores de pagamento (por exemplo, ``invoice.payment_failed``) para atualizar sua tabela ``Tenant`` de acordo.

Subdomínios (Módulo 11)

Usando o Middleware do Next.js, aprenderemos como reescrever as requisições para que os inquilinos possam ter sua própria URL corporativa, ex: ``empresaA.meusaas.com`` e ``empresaB.meusaas.com``.

Implantação e CI/CD (Módulos 15 e 16)

Migraremos seu PostgreSQL Docker local para um provedor de nuvem como Neon ou Supabase. Empurraremos nosso código Next.js para a Vercel, o que construirá e distribuirá automaticamente seu SaaS em escala global.

FULLSTACK SAAS ARCHITECT

Certificação concedida a:

[DEVELOPER_NAME]

Por ter dominado o design e desenvolvimento de plataformas Multi-Tenant, integrando bancos de dados relacionais, autenticação robusta e gateways de pagamento recorrente para levar produtos de 0 à Produção.

TIMESTAMP: 2026-08-12 // SYS_ADMIN_SIGNATURE